

Informe Técnico de Observaciones — App Integra ESAVI

Proyecto	App Integra ESAVI — <code>app-integra-esavi</code> rama <code>main</code>
Fecha	14 de junio de 2026
Elaborado por	Equipo de Desarrollo
Alcance	<code>src/</code> · <code>public/</code> · configuración de entorno y build

Informe de observaciones técnicas identificadas mediante análisis estático del frontend. Cada observación incluye referencia al archivo y línea de código correspondiente. Se excluyen del análisis los módulos **calidad**, **dashboard** y **gaceta**.

OBS-01 · Errores funcionales en componentes de vista

Archivos principales: `src/pages/esavis/ESAVISList.tsx` · `src/dataProviders/esavis.dataprovider.ts`

Carácter suelto renderizado como texto en la tabla de ESAVIs. En `ESAVISList.tsx` línea 137, una `s` literal aparece directamente en el JSX entre el cierre del primer `<FunctionField>` de fecha y el inicio del segundo. Al renderizar, React interpreta ese carácter como nodo de texto y lo muestra visible en cada fila de la tabla, produciendo contenido basura en producción. Es el error de mayor impacto visual inmediato.

Dos columnas con etiqueta idéntica "Código Origen". Las líneas 102 y 110 del mismo archivo definen dos `<FunctionField>` consecutivos con `label="Código Origen"`. El primero muestra el sistema de origen (VIGIFLOW / DHIS2) y el segundo el código del caso. React Admin usa el `label` como encabezado de columna; al ser idénticos, la tabla presenta encabezados duplicados, impidiendo al usuario distinguir qué columna contiene qué información.

Mensaje de error incorrecto en `getOne`. El método `getOne` de `esavis.dataprovider.ts` (línea 78) lanza `throw new Error("Error al obtener la lista de ESAVIs")` cuando falla la recuperación de un registro individual. El mensaje está copiado del método `getList` y describe una operación distinta, dificultando la identificación del origen del error durante la depuración.

`console.log` activos en el render de cada fila. Dentro del `render` del `<FunctionField>` de `Id` (línea 72 de `ESAVISList.tsx`), se ejecuta `console.log("Record :::", record)`. Al estar ubicado dentro del `render`, este log se dispara en cada re-render de cada fila visible en pantalla, volcando datos personales de pacientes (id, identificación, nombre) en la consola del navegador sin restricción de entorno. Los métodos `getList` y `getOne` del data provider (líneas 46 y 66) también tienen `console.log("params", params)` activos con idéntico impacto.

OBS-02 · Seguridad, exposición de datos y resiliencia

Archivos: `src/dataProviders/esavis.dataprovider.ts` · `src/utils/env_utils.ts` · `src/dataProviders/axios.client.ts`

getMany descarga la totalidad de registros para filtrar en memoria. El método `getMany` de `esavis.dataprovider.ts` (línea 89) invoca el endpoint `/findAllPaginated` sin parámetros de paginación ni filtro, recuperando la totalidad de notificaciones disponibles en la base de datos y filtrando localmente por los `ids` solicitados mediante `Array.filter()`. React Admin invoca `getMany` en operaciones de referencias cruzadas y selecciones múltiples; en un entorno con miles de notificaciones ESAVI este patrón genera timeouts de red, consumo excesivo de memoria en el navegador y carga innecesaria sobre el servidor. El mismo endpoint acepta el parámetro `ids` en su contrato POST, por lo que el filtrado debería delegarse al backend.

Variables de entorno críticas sin validación al iniciar. `env_utils.ts` consume directamente `import.meta.env` sin verificar la presencia de ninguna variable requerida. Cuando falta `VITE_INTEGRA_ESAVI_API_URL`, `VITE_APP_API_KEY` o cualquiera de las variables de Keycloak, la aplicación arranca sin emitir ningún error visible; las llamadas a la API fallan silenciosamente con errores de red genéricos que dificultan el diagnóstico. Esta ausencia de validación es especialmente crítica en despliegues a nuevos entornos donde una variable mal configurada puede pasar desapercibida.

Interceptores de Axios declarados sin lógica. `axios.client.ts` registra interceptores de request y response (líneas 13–31) que únicamente hacen pass-through del parámetro recibido. No implementan renovación de token, manejo de errores HTTP (401, 500), ni registro de auditoría. Declarar interceptores vacíos genera ruido sin valor, y priva al cliente de un punto centralizado donde implementar comportamiento transversal —como redirigir a login ante un 401 o mostrar un mensaje global ante un 503— sin duplicar lógica en cada data provider.

OBS-03 · Calidad de código y uso de TypeScript

Archivos: `src/contexts/AuthContext.tsx` · `src/authorization.utils.tsx` · `src/pages/vacunometro/vacunometroList.tsx` · `src/pages/esavis/ESAVISList.tsx`

Nombre de archivo con espacio rompe en entornos Linux y Docker. El archivo `src/contexts/AuthContext.tsx` contiene un espacio antes de la extensión. En sistemas de archivos sensibles a mayúsculas y minúsculas —como los de los contenedores Docker que ejecutan la aplicación en producción— este nombre puede no resolverse correctamente, produciendo fallos de importación en tiempo de build. Todos los archivos que lo importan (entre ellos `App.tsx`, `authorization.utils.tsx` y `layout/CustomMenu.tsx`) referencian la ruta con el espacio. El archivo debe renombrarse a `AuthContext.tsx` y actualizarse todas sus importaciones.

Tipado `any` generalizado en componentes y contextos críticos. `AuthContext.tsx` declara `realm_access: any | null` y `resource_access: any | null` pese a que las interfaces `RealmAccess` y

`ResourceAccess` están definidas en el mismo archivo y describen exactamente la estructura esperada. El componente `AuthProvider` tipifica sus `children` como `any` en lugar de `ReactNode`. En `authorization.utils.tsx`, el componente `<Authorize>` recibe sus tres props (`allowedRoles`, `deniedRoles`, `children`) tipificadas como `any`, dejando sin validación estática el contrato de una pieza central del sistema de control de acceso. Este patrón se replica en los data providers, donde los datos de retorno de la API se castean sistemáticamente como `any[]`.

Componente `ListActions` definido dentro del render de `VacunometroList`. En `vacunometroList.tsx` línea 24, `ListActions` se declara como función dentro del cuerpo de `VacunometroList`. React crea una nueva referencia de componente en cada render del padre, provocando que React Admin desmonte y vuelva a montar el toolbar en cada actualización de estado—incluyendo cada apertura y cierre del diálogo de sincronización. El componente debe extraerse al nivel del módulo como función independiente y recibir `open` y `setOpen` como props explícitas.

Estado `isHover` declarado sin ningún uso efectivo. `ESAVISList.tsx` declara `const [isHover, setIsHover] = useState(false)` y asigna los handlers `onMouseEnter / onMouseLeave` a un `<label>`, pero el valor `isHover` no condiciona ningún render, estilo ni efecto en ningún punto del componente. Es código muerto que añade complejidad sin propósito.

OBS-04 · Infraestructura, pruebas y deuda técnica

Archivos: `package.json` · `pnpm-lock.yaml` · `package-lock.json` · `src/App.tsx` · `src/setupTests.js`

Mezcla de gestores de paquetes en el repositorio. `package-lock.json` (npm) y `pnpm-lock.yaml` (pnpm) coexisten en el repositorio. El proyecto está configurado para pnpm: el `.npmrc` define `shamefully-hoist`, `pnpm-workspace.yaml` especifica el workspace y el `Dockerfile.dev` instala dependencias con `pnpm install`. La presencia simultánea de `package-lock.json` invita a instalar con npm en algunos entornos, produciendo árboles de dependencias distintos e incompatibilidades entre entornos de desarrollo y producción. El `package-lock.json` debe eliminarse y bloquearse su generación añadiendo `engine-strict=true` en `.npmrc`.

Ausencia total de pruebas con infraestructura ya disponible. La dependencia `@testing-library/react` está instalada y `src/setupTests.js` está configurado, pero no existe ningún archivo `.test.tsx` ni `.spec.ts` en el proyecto. Los módulos de mayor riesgo funcional sin ninguna cobertura son: `authorization.utils.tsx` (control de acceso basado en roles), la función `ocultarInformacion` (privacidad de datos de pacientes), `esavis.dataprovider.ts` (paginación, filtros y mapeo de respuestas) y `env_utils.ts` (validación de configuración). La ausencia de pruebas en el componente `<Authorize>` implica que una regresión en la lógica de roles podría exponer funcionalidades protegidas sin ningún mecanismo de detección automática.

TypeScript 4.9 desactualizado frente al resto del stack. El proyecto usa TypeScript 4.9.4, mientras que Vite 5, React 18 y React Admin 5 son compatibles con TypeScript 5.x. La versión 5 introduce `const` type parameters, resolución mejorada de tipos condicionales y mejor inferencia en patrones

genéricos de React, funcionalidades que permitirían eliminar varios de los castings a `any` identificados en OBS-03.

Código comentado obsoleto en `App.tsx`. Las líneas 23, 35 y 36 contienen la importación de `createHashHistory` y su instanciación, comentadas desde versiones anteriores. La prop `history` del componente `<Admin>` también aparece comentada en línea 38. Si la migración a hash routing fue descartada, estas líneas deben eliminarse; de lo contrario, deben registrarse como tarea pendiente en el sistema de seguimiento del proyecto.

Tabla consolidada de observaciones

ID	Área	Descripción	Severidad
OBS-01a	Vista	Carácter <code>s</code> suelto visible en cada fila de la tabla de ESAVIs en producción	Alta
OBS-01b	Vista	Dos columnas con <code>label="Código Origen"</code> idéntico en <code>ESAVISList</code>	Alta
OBS-01c	Código	Mensaje de error de <code>getOne</code> describe la operación <code>getList</code>	Media
OBS-01d	Seguridad	<code>console.log</code> con datos de pacientes activo en render de producción	Alta
OBS-02a	Rendimiento	<code>getMany</code> descarga todos los registros en memoria para filtrar por <code>ids</code>	Alta
OBS-02b	Resiliencia	Variables de entorno críticas sin validación al iniciar la aplicación	Media
OBS-02c	Código	Interceptores Axios declarados vacíos sin lógica transversal	Baja
OBS-03a	Estructura	Nombre de archivo <code>AuthContext.tsx</code> con espacio; falla en Docker/Linux	Alta
OBS-03b	TypeScript	Tipado <code>any</code> en <code>AuthContext</code> , <code><Authorize></code> y <code>data providers</code>	Media
OBS-03c	Rendimiento	<code>ListActions</code> definido dentro de <code>VacunometroList</code> ; desmonte en cada render	Media
OBS-03d	Código	Estado <code>isHover</code> declarado y asignado pero sin efecto alguno en el render	Baja
OBS-04a	Infraestructura	<code>package-lock.json</code> y <code>pnpm-lock.yaml</code> coexisten; riesgo de árboles divergentes	Media
OBS-04b	Calidad	Sin pruebas implementadas en módulos de control de acceso y privacidad de datos	Media
OBS-04c	Deuda técnica	TypeScript 4.9 desactualizado; stack compatible con TS 5.x	Baja
OBS-04d	Código	Importación y uso de <code>createHashHistory</code> comentados desde versiones anteriores	Baja